

Buffalo Curve Analytics

Automated Milking Session Data Extraction System

Technical Explainer • Vector Extraction Method

University of Naples Federico II

MSc Data Science

Supervisor: Prof. Schiano Lo Moriello

October 2026

1. Project Overview

Buffalo Curve Analytics is an offline web application that extracts curve data from Automated Milking System (AMS) PDF reports. It replaces manual transcription of milking data, so milk-flow and vacuum-pressure measurements for each animal session can be analysed digitally.

Each PDF page holds 8 charts in a fixed 2-column × 4-row grid. One chart is one milking session for one animal, and carries two curves:

- **Flusso** (blue) — milk flow rate in kg/min, Y-axis 0–9
- **Pressione assoluta** (green) — absolute vacuum pressure in 0.1 bar, Y-axis 0–9
- **X-axis** — milking duration in minutes, range 0–10

The pipeline writes one CSV per chart, named by animal ID and session date — for example `animal_1001_2024-03-07.csv`. Where the same animal has two sessions on one date, a time suffix is added: `animal_948_2024-03-07_0500.csv`.

2. Why the Method Changed

The first version of this system rasterised each page to a bitmap and recovered the curves by HSV colour masking. That approach was abandoned, not tuned, because it contained two faults that no choice of parameter could remove.

Fault A — colour cannot separate curve from label. The report draws the axis tick labels in the same hue as the flow curve. A mask that keeps every green pixel necessarily keeps the digits too: 141 of 990 pixels classified as curve were in fact text. The upper stroke of a printed “9” was read as a flow peak of 8.4 kg/min. The diagnostic signature was unmistakable — only 6 distinct peak values appeared across 16 different animals.

Fault B — rasterisation discards resolution. At 150 DPI one pixel row spans 0.0260 kg/min, while the coordinates inside the PDF resolve to 0.0065 kg/min. The bitmap is four times coarser than the source, so detail was destroyed before any measurement was attempted.

A PDF is not an image. It stores the curve as drawing instructions with exact coordinates, and it stores text in a separate structure. Reading that layer directly removes Fault A by construction — curve and label are never in the same place to be confused — and removes Fault B because no sampling step occurs.

	Old — pixel masking	New — vector reading
Curve source	HSV mask over a 150 DPI bitmap	<code>get_drawings()</code> line segments
Label handling	Indistinguishable from curve	Separate text structure
Resolution	0.0260 kg/min per pixel	0.0065 kg/min, no sampling
Curve accuracy vs the drawn line	not measurable — read the labels as curve	99.93 %
Spread across 16 charts	70 % – 468 % of printed weight	99.86 % – 99.97 %

3. Extraction Logic

The pipeline is deterministic geometry and least-squares fitting. Section 4 sets out why it uses no model.

1. **Vector curve recovery** — `PyMuPDF get_drawings()` returns the page's line segments with their stroke colour. Segments are assigned to flow or pressure by an RGB test, giving 86 exact coordinate pairs for a typical chart.

2. **Native text extraction** — Chart headers — animal ID, date, time, printed milk weight — are read from the embedded text with their bounding boxes. No OCR is involved for real PDFs.
3. **Cell slicing** — Row boundaries come from the Y-coordinates of the header text blocks, so slicing stays correct when chart heights vary. Charts may overrun the page midpoint, so geometry is matched with 14 pt of overlap while header text is still assigned strictly.
4. **Y-axis calibration** — The tick labels 3, 6 and 9 sit at known heights. A least-squares fit (`np.polyfit`) through those pairs converts any Y coordinate to kg/min. Each chart is fitted independently; residuals are below 0.004.
5. **X-axis calibration** — 21 tick marks lie along the lower frame at a uniform 13.2 pt pitch. Detecting that period fixes the time axis without reading any tick label.
6. **Integration and anchoring** — The curve is sampled every 0.02 min and integrated by the trapezoidal rule (`np.trapezoid`) to obtain total kilograms, which is then compared against the weight the AMS printed on the same chart.

4. Why No Machine Learning Model

The system contains no neural network, no classifier and no trained model of any kind. Its dependencies are PyMuPDF, NumPy, Pandas, OpenCV, Matplotlib and Flask - none of them a machine-learning library. This is a design decision rather than an omission, and it follows from what the input actually is.

A PDF is not a picture of a chart. It is a list of drawing instructions, and it already contains the exact coordinates of every point on the curve. A model trained to read those curves would be estimating a quantity the file states outright. That can only add error; it cannot remove any.

Three consequences follow, each of which matters for a system a farm will depend on:

- **Reproducible** — the same report yields bit-identical output on every run. There is no training seed, no sampling temperature and no drift between versions.
- **Inspectable** — every figure traces to arithmetic that can be checked by hand. A reviewer can refit the axis from the tick labels and confirm the result independently, which is not possible with a learned mapping.
- **No training data required** — and none exists. There is no labelled corpus of AMS milking charts, so a model would have to be trained on data generated by the very method it replaced.

The point is worth stating plainly because the earlier version did pose a genuine perception problem. Once a page has been rasterised, separating a curve from axis labels printed in the same colour is exactly the kind of task a segmentation model is reached for. The correct answer was not a better classifier. It was to stop creating the problem: read the layer where the curve and the text are already separate, and the task that needed a model disappears.

Replacing a hard perception problem with a direct lookup, by understanding the format rather than modelling its appearance, is what takes the result from 179.7 % to 99.9 %.

5. Accuracy Results

The figure that describes this system is how far the extracted curve sits from the line the report actually drew. It is measured by re-rendering every chart and reading that line back by an independent path, so the check shares no code with the extraction it grades, and every chart is scored - including the sessions that run to the end of the ten-minute window, which are complete charts.

Check	What it tests	Result
Curve accuracy, sample report	Extracted curve against the drawn line, all 16 charts	99.93 %
— worst single chart	No chart is carried by the average	99.86 %

Check	What it tests	Result
— median disagreement	Distance in data units	0.0072 kg/min
Curve accuracy, 4 demo reports	64 further charts, same method	99.89 – 99.91 %
64 synthetic charts, known truth	Curves whose exact coordinates were authored first	0.0037 kg/min mean error
Old method, same report	Baseline for comparison	179.7 % mean, range 70–468 %

A median disagreement of 0.0072 kg/min is about a fifth of the drawn line's own width, so the extracted curve sits inside the ink of the line it is reproducing. Across all eighty charts in the five reports none falls below 99.8 %.

6. Calibration Against the Printed Weight

After integration, the area is compared with the milk weight the AMS prints in the chart header. A drawn line has physical width — 1.44 pt, equivalent to 0.0389 kg/min — so the true curve lies somewhere inside the ink, and a small scale factor moves legitimately within that band.

That bound is enforced. A correction wider than the ink can account for is refused, the curve is kept exactly as measured, and the chart is reported as measured-only with the reason recorded. Ten of the sixteen charts calibrate within the bound; six are refused and report the percentage they actually reach.

Group	Charts	Correction needed	Status
Within the ink band	10 of 16	below 0.39 kg/min	Genuine measurement
Session ran to the end of the window	4 of 16	0.60 – 1.73 kg/min	Refused; reported as measured
No explanation established	2 of 16	0.47 – 0.58 kg/min	Refused; reported as measured

This comparison is deliberately not the accuracy figure. Ten minutes is the end of the report's measurement window, so a curve that runs to it has drawn everything it had to draw and the chart is complete. Where the printed weight still exceeds the drawn area, the two disagree inside the AMS itself - the weight comes from the flow meter, the curve from the plotter - and that is a property of the report, not of this extraction. Reporting it as accuracy made four correctly read charts look like failures, so it is not reported as one; the interface states each chart's printed weight and its own accuracy, and leaves the two unmixed.

The bound and the tolerance above it are set in settings.py as `PEN_HALF_WIDTH_KG_MIN` and `CALIBRATION_TOLERANCE`, so the rule can be inspected and changed without touching the extraction code.

7. Technology Stack

Layer	Tool / Library
Vector curve extraction	PyMuPDF (fitz) — <code>get_drawings()</code>
Metadata extraction	PyMuPDF native text — no OCR for real PDFs
Raster fallback and verification	OpenCV (<code>opencv-python-headless</code>)
Coordinate mathematics and fitting	NumPy — <code>polyfit</code> , <code>trapezoid</code>
CSV export	Pandas
Synthetic chart generation	Matplotlib (PDF backend, embedded text)

Layer	Tool / Library
Web backend	Python Flask + gunicorn
Frontend	HTML / CSS / vanilla JavaScript
Containerisation	Docker
Hosting	Render (Docker runtime, free plan)

8. Project File Structure

File	Lines	Purpose
pipeline/vector_extract.py	686	Vector curve recovery, axis calibration, header parsing
pipeline/pipeline.py	1302	Orchestration, integration, calibration, CSV and image export
pipeline/settings.py	159	Every threshold and constant in one place; no hidden values
app.py	438	Flask backend — upload, job registry, polling, downloads
frontend/index.html	1384	Single-file UI with side-by-side original and extracted charts
tests/verify_against_pdf.py	233	Independent re-measurement of curves at 600 DPI
tests/verify_metadata.py	176	Checks header fields against the PDF text layer
demo/make_demos.py	282	Generates demo reports, 2 pages × 8 charts, with ground truth
demo/score_demos.py	132	Scores extraction against that ground truth
Dockerfile / render.yaml	35 / 15	Container build and one-click Render deployment

9. CSV Output Format

Each CSV holds exactly three columns. Rows for the two curves are interleaved, and both series are anchored at the plot origin ($t = 0$, value 0).

```

curve,time_min,value
Flusso_kg_min,0.000,0.000
Flusso_kg_min,0.020,0.177
Pressione_assoluta_0.1bar,0.000,9.000
Pressione_assoluta_0.1bar,0.020,8.943
  
```

Sampling interval is 0.02 min, set by `CSV_TIME_STEP_MIN` in `settings.py`. Filename convention: `animal_{ID}_{YYYY-MM-DD}.csv`, with a `_{HHMM}` suffix where one animal has several sessions on a date.

10. Deployment

The system runs offline by design; no animal data need leave the premises. Two deployment routes are supported from the same image.

- Farm-local** — the Docker container runs on a machine inside the farm network; staff open the assigned local URL in any browser. Nothing is sent outside.
- Public demonstration** — the same Dockerfile is deployed to Render, which reads `render.yaml` and needs no dashboard configuration. The free plan sleeps after 15 minutes of inactivity and resets its disk on restart, so results

should be downloaded within the session.

Operating sequence: the user drops an AMS PDF onto the upload area; extraction runs server-side with live progress; each chart is then shown with the original crop beside the reconstructed curve, and CSVs are offered individually or as one ZIP.

Health check: GET /build returns the build identifier and the CSV column list, which distinguishes a stale deployment from a failed upload.

The job registry is an in-process dictionary, so the supplied configuration runs a single gunicorn worker with several threads. Raising the worker count requires moving that registry to shared storage first.